```tsx
/**
 * components/ui/data-freshness-badge.tsx
 *
 * Recommendation #10 — A prediction made on 6-hour-old telemetry looks identical
 * to one made on real-time data. This badge surfaces data age on any component
 * that shows a prediction or risk score, so operators can judge reliability.
 *
 * Returns null when data is fresh — zero visual noise in the happy path.
 *
 * Usage:
 *   <DataFreshnessBadge timestamp={equipment.lastTelemetryAt} />
 *
 *   // Custom thresholds for equipment that should report every 5 minutes:
 *   <DataFreshnessBadge
 *     timestamp={latestReading}
 *     staleThresholdMinutes={10}
 *     criticalThresholdMinutes={30}
 *   />
 */

import { Clock, AlertTriangle, WifiOff } from 'lucide-react';
import { formatDistanceToNow } from 'date-fns';
import { Badge } from '@/components/ui/badge';
import {
  Tooltip,
  TooltipContent,
  TooltipProvider,
  TooltipTrigger,
} from '@/components/ui/tooltip';
import { cn } from '@/lib/utils';
import { getDataFreshness, type FreshnessLevel } from '@/lib/severity';

// ─── Props ──────────────────────────────────────────────────────────────────

interface DataFreshnessBadgeProps {
  timestamp: Date | string | null | undefined;
  /**
   * Minutes after which data is considered "stale" (amber warning).
   * Default: 60 min. Override for equipment with faster reporting cadence.
   */
  staleThresholdMinutes?: number;
  /**
   * Minutes after which data is considered "critical" (red warning).
   * Default: 180 min.
```

```
   */
  criticalThresholdMinutes?: number;
  /** Size variant */
  size?: 'sm' | 'default';
  className?: string;
  'data-testid'?: string;
}

// ─── Config per freshness level ──────────────────────────────────────

const levelConfig: Record<
  Exclude<FreshnessLevel, 'fresh'>,
  {
    icon: typeof Clock;
    className: string;
  }
> = {
  stale: {
    icon: Clock,
    className:
      'border-amber-400/60 text-amber-600 dark:text-amber-400 bg-amber-50 dark:bg
  },
  critical: {
    icon: AlertTriangle,
    className:
      'border-red-400/60 text-red-600 dark:text-red-400 bg-red-50 dark:bg-red-950
  },
  unknown: {
    icon: WifiOff,
    className: 'border-border text-muted-foreground',
  },
};

// ─── Component ──────────────────────────────────────────────────────────

export function DataFreshnessBadge({
  timestamp,
  staleThresholdMinutes = 60,
  criticalThresholdMinutes = 180,
  size = 'sm',
  className,
  'data-testid': testId,
}: DataFreshnessBadgeProps) {
  const freshness = getDataFreshness(timestamp, staleThresholdMinutes, criticalTh

  // Fresh data → render nothing. Zero noise in the normal case.
  if (freshness.level === 'fresh') return null;
```

```
  const cfg = levelConfig[freshness.level as Exclude<FreshnessLevel, 'fresh'>];
  const Icon = cfg.icon;

  const timeAgo = timestamp
    ? formatDistanceToNow(new Date(timestamp), { addSuffix: true })
    : null;

  return (
    <TooltipProvider>
      <Tooltip>
        <TooltipTrigger asChild>
          <Badge
            variant="outline"
            className={cn(
              'cursor-help gap-1',
              size === 'sm' ? 'h-5 text-xs px-1.5' : 'h-6 text-xs px-2',
              cfg.className,
              className,
            )}
            data-testid={testId}
          >
            <Icon className={size === 'sm' ? 'h-3 w-3' : 'h-3.5 w-3.5'} />
            {freshness.label}
          </Badge>
        </TooltipTrigger>
        <TooltipContent className="max-w-xs space-y-1">
          <p className="font-medium text-sm">{freshness.description}</p>
          {timeAgo && (
            <p className="text-xs text-muted-foreground">
              Last reading received {timeAgo}
            </p>
          )}
          {freshness.level === 'critical' && (
            <p className="text-xs text-red-500 dark:text-red-400 font-medium">
              Check sensor connectivity before acting on this prediction.
            </p>
          )}
        </TooltipContent>
      </Tooltip>
    </TooltipProvider>
  );
}
```